

Documentação de Projeto para o Sistema Hermes

Versão: 1.0 Autor: Arthur Miranda Sales Data: 08/06/2026

Tabela de Conteúdo

1. Introdução
2. Modelos de Usuário e Requisitos
 - 2.1 Descrição de Atores
 - 2.2 Modelo de Casos de Uso
 - 2.3 Diagrama de Sequência do Sistema (SSD) e Contratos de Operação
3. Modelos de Projeto
 - 3.1 Arquitetura
 - 3.2 Diagrama de Componentes e Implantação
 - 3.3 Diagrama de Classes
 - 3.4 Diagramas de Sequência
 - 3.5 Diagramas de Comunicação
 - 3.6 Diagramas de Estados
4. Modelos de Dados

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Arthur Miranda Sales	08/06/2026	Criação do documento a partir da análise factual do projeto Hermes	1.0

1. Introdução

Este documento descreve o projeto do **Sistema Hermes**, uma plataforma de gestão para vendedores (*sellers*) que operam na Amazon. Seu propósito é registrar, de forma acadêmico-profissional, os modelos de requisitos e de projeto que sustentam a implementação: atores, casos de uso, arquitetura, classes de domínio, fluxos de sequência/comunicação, máquinas de estado e o modelo de dados relacional.

O domínio do Hermes é a **operação de venda na Amazon**. O sistema conecta-se às APIs oficiais da Amazon (**SP-API** para pedidos, inventário, financeiro e listagem; e **Amazon Ads API** para publicidade), sincroniza dados da loja, calcula métricas e margens, automatiza precificação e campanhas de anúncios, gere assinaturas/pagamentos e oferece uma extensão para o navegador Chrome voltada à análise de produtos.

Tecnicamente, o Hermes é um **Monólito Modular** em **.NET 9 / ASP.NET Core**, organizado em nove módulos com **Clean Architecture** por módulo, **CQRS** via **MediatR** e padrão **Outbox** para eventos de integração. A persistência usa **SQL Server** com **EF Core 9**, incluindo estratégia **multi-tenant banco-por-loja**. As seções seguintes detalham cada modelo.

Nota de fidelidade ao projeto: onde versões preliminares da documentação divergiam da arquitetura adotada, prevaleceram as decisões do projeto: a mensageria WhatsApp é a **W-API** e o SGBD é **SQL Server**.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Atores humanos

Ator	Descrição
Vendedor Amazon	Usuário principal, autenticado via JWT e com a role assinante. Conecta lojas, cria/gerencia campanhas de Ads, consulta métricas, configura precificação, reviews e tributação. O acesso a features é controlado por <i>claims</i> de plano (PolicyExtension, PolicyMining, PolicyAnalytics).
Administrador de Conta	Membro de uma <i>billing account</i> com papel Owner, Admin ou Member (enum BillingAccountMemberRole). Administra assinaturas, planos e cobrança da conta de faturamento.

Atores externos (sistemas integrados)

Ator externo	Papel no sistema
Amazon SP-API	OAuth de conexão da loja; pedidos, inventário, financeiro, listagem, notificações.
Amazon Ads API	OAuth de Ads; campanhas, ad groups, keywords, targets, relatórios.
AWS SQS / EventBridge	Entrega assíncrona das notificações da SP-API.
Stripe	Pagamentos e assinaturas (módulo Payments), via webhooks idempotentes.
PagSeguro / Eduzz / Kiwify	Gateways de pagamento legados (módulo GatewayPay).
Keepa	Histórico e preços de mercado de produtos.
Melhor Envio	Cálculo de frete.
Infosimples / INPI	Consulta de marca/trademark.
Azure Blob / AI Vision / Document Intelligence	Auditoria de pedidos e processamento de catálogos de fornecedor.
Brevo	E-mails transacionais.
W-API (WhatsApp)	Notificações por WhatsApp.
n8n	Webhook de notificações de suporte.
Google OAuth	Login externo.
Redis	Cache de extensão/produtos/mineração.

2.2 Modelo de Casos de Uso

```
@startuml
    casos-de-uso
    left to right direction
    skinparam packageStyle rectangle
    title Hermes - Modelo de Casos de Uso

    actor "Vendedor Amazon\n(role: assinante)" as Seller
    actor "Administrador\nde Conta" as Admin

    actor "Amazon SP-API" as SPAPI
    actor "Amazon Ads API" as ADS
    actor "AWS SQS /\nEventBridge" as SQS
    actor "Stripe" as Stripe
    actor "Gateways legados\n(PagSeguro/Eduzz/Kiwify)" as Legacy
```

```

actor "Keepa" as Keepa
actor "Infosimples /\nINPI" as INPI
actor "W-API\n(WhatsApp)" as WApi

rectangle "Sistema Hermes" {
  usecase "UC-01 Conectar loja Amazon (OAuth SP-API)" as UC01
  usecase "UC-02 Conectar conta Amazon Ads" as UC02
  usecase "UC-03 Desconectar loja" as UC03
  usecase "UC-04 Criar campanha Amazon Ads" as UC04
  usecase "UC-05 Gerenciar campanhas/keywords/targets" as UC05
  usecase "UC-06 Relatorios e metricas de Ads" as UC06
  usecase "UC-07 Estrategias automaticas de Ads" as UC07
  usecase "UC-08 Sincronizar pedidos e financeiro" as UC08
  usecase "UC-09 Consultar metricas de vendas" as UC09
  usecase "UC-10 Gerenciar inventario/custos FIFO" as UC10
  usecase "UC-11 Precificacao automatica (Repricing)" as UC11
  usecase "UC-12 Gestao de listagem" as UC12
  usecase "UC-13 Automacao de reviews" as UC13
  usecase "UC-14 Configuracao tributaria da loja" as UC14
  usecase "UC-15 Identidade e acesso" as UC15
  usecase "UC-16 Pagamentos Stripe" as UC16
  usecase "UC-17 Pagamentos legados" as UC17
  usecase "UC-18 Catalogo, favoritos e fornecedores" as UC18
  usecase "UC-19 Extensao Chrome: analise/tarifas/marca" as UC19
  usecase "UC-20 Planner (unit economics)" as UC20
}

```

```

Seller --> UC01
Seller --> UC02
Seller --> UC03
Seller --> UC04
Seller --> UC05
Seller --> UC06
Seller --> UC07
Seller --> UC09
Seller --> UC10
Seller --> UC11
Seller --> UC12
Seller --> UC13
Seller --> UC14
Seller --> UC15
Seller --> UC16
Seller --> UC18
Seller --> UC19
Seller --> UC20
Admin --> UC16
Admin --> UC17

```

```

UC01 --> SPAPI
UC03 --> SPAPI
UC08 --> SPAPI
UC08 --> SQS
UC09 --> SPAPI
UC02 --> ADS
UC04 --> ADS
UC05 --> ADS
UC06 --> ADS
UC07 --> ADS
UC16 --> Stripe
UC17 --> Legacy
UC19 --> Keepa
UC19 --> INPI

```

UC15 --> WApi
@endum1

Tabela de casos de uso

ID	Caso de uso	Ator(es) principal(is)
UC-01	Conectar loja Amazon (OAuth SP-API)	Vendedor, Amazon SP-API
UC-02	Conectar conta Amazon Ads	Vendedor, Amazon Ads API
UC-03	Desconectar loja	Vendedor, Amazon SP-API
UC-04	Criar campanha Amazon Ads (Sponsored Products)	Vendedor, Amazon Ads API
UC-05	Gerenciar campanhas/ad groups/targets/keywords	Vendedor, Amazon Ads API
UC-06	Relatórios e métricas Amazon Ads	Vendedor, Amazon Ads API
UC-07	Estratégias automáticas de Ads	Vendedor, Amazon Ads API
UC-08	Sincronização de pedidos e financeiro	Amazon SP-API, AWS SQS
UC-09	Consultar métricas/consolidações de vendas	Vendedor
UC-10	Gerenciar inventário/custos (FIFO)	Vendedor
UC-11	Precificação automática (Repricing)	Vendedor
UC-12	Gestão de listagem (Listings Items API)	Vendedor, Amazon SP-API
UC-13	Automação de reviews	Vendedor, Amazon SP-API
UC-14	Configuração tributária da loja	Vendedor
UC-15	Identidade e acesso	Vendedor, Google OAuth
UC-16	Pagamentos Stripe	Vendedor, Administrador, Stripe
UC-17	Pagamentos legados	Administrador, PagSeguro/Eduzz/Kiwify
UC-18	Catálogo, favoritos e fornecedores	Vendedor
UC-19	Extensão Chrome: análise/tarifas/marca	Vendedor, Keepa, INPI
UC-20	Planner (unit economics)	Vendedor

2.3 Diagrama de Sequência do Sistema (SSD) e Contratos de Operação

SSD — UC-01 (Conectar loja Amazon)

```
@startuml
    actor Vendedor
    participant Sys as "Sistema Hermes"
    participant SPAPI as "Amazon SP-API"

    Vendedor->>Sys: iniciarConexaoAmazon(userId)
    activate Sys
    Sys-->>Vendedor: redirectUrl (Seller Central consent)
    deactivate Sys

    Vendedor->>SPAPI: autoriza consentimento
```

```

SPAPI --> Vendedor : redirect callback (spapi_oauth_code, state)

Vendedor -> Sys : autorizarAmazon(code, sellingPartnerId, mwsAuthToken, state)
activate Sys
Sys -> SPAPI : exchangeAuthorizationCode(code)
SPAPI --> Sys : refreshToken, accessToken
Sys -> SPAPI : getSellerInfo(refreshToken)
SPAPI --> Sys : sellerInfo
Sys -> SPAPI : ensureSubscription(ORDER_CHANGE)
SPAPI --> Sys : subscriptionId
Sys --> Vendedor : redirect https://portal.hermess.app
deactivate Sys
@enduml

```

SSD — UC-02 (Conectar conta Amazon Ads)

```

@startuml ssd-uc02
title SSD UC-02 - Conectar conta Amazon Ads
actor Vendedor
participant ":Sistema Hermes" as Sys
participant "Amazon Ads API" as ADS

Vendedor -> Sys : iniciarConexaoAds(userId, storeId)
activate Sys
Sys --> Vendedor : redirectUrl (consent Amazon Ads)
deactivate Sys

Vendedor -> ADS : autoriza consentimento
ADS --> Vendedor : redirect callback (code, state)

Vendedor -> Sys : autorizarAmazonAds(code, state)
activate Sys
Sys -> ADS : exchangeAuthorizationCode(code)
ADS --> Sys : refreshToken
Sys -> ADS : listProfiles(refreshToken)
ADS --> Sys : profiles[]
Sys -> Sys : seleciona profile por marketplace da loja
Sys --> Vendedor : conexao Ads ativa (AmazonAdsConnection)
deactivate Sys
@enduml

```

SSD — UC-03 (Desconectar loja)

```

@startuml ssd-uc03
title SSD UC-03 - Desconectar loja
actor Vendedor
participant ":Sistema Hermes" as Sys
participant "Amazon SP-API" as SPAPI

Vendedor -> Sys : desconectarLoja(storeId)
activate Sys
Sys -> Sys : localiza AmazonConnects da loja
Sys -> Sys : Disconnect() (IsActive=false, limpa tokens)
Sys -> SPAPI : cancela subscriptions de notificacao (best-effort)
SPAPI --> Sys : ok
Sys --> Vendedor : loja desconectada
deactivate Sys
@enduml

```

Contrato de Operação — UC-01

Contrato	
Operação	<code>Task<string> autorizarAmazon(string code, string sellingPartnerId, string mwsAuthToken, string state)</code>
Referências cruzadas	UC-01 (alimenta UC-08 e UC-09)
Pré-condições	Usuário autenticado; <code>userId</code> codificado no <code>state</code> (Base64Url) gerado em <code>InitiateAmazon</code> ; a Amazon retornou <code>spapi_oauth_code</code> e <code>state</code> válidos; limite <code>StoreConnection:MaxStoresPerUser</code> respeitado.
Pós-condições	Código trocado por tokens (<code>ExchangeAuthorizationCodeAsync</code>); <code>AmazonConects</code> criado/atualizado (<code>upsert</code>) com <code>refresh/access token</code> → <code>connectId</code> ; <code>Stores</code> criada/recuperada vinculada a <code>userId</code> e <code>connectId</code> ; <code>subscription</code> de notificações <code>ORDER_CHANGE</code> registrada na SP-API (best-effort); retorna URL de redirect <code>https://portal.hermess.app</code> .

Contrato de Operação — UC-02

Contrato	
Operação	<code>Task<Result> autorizarAmazonAds(string code, string state)</code>
Referências cruzadas	UC-02 (pré-requisito de UC-04, UC-05, UC-06, UC-07)
Pré-condições	Usuário autenticado; loja (<code>storeId</code>) existente e conectada via UC-01; a Amazon Ads retornou <code>code</code> e <code>state</code> válidos.
Pós-condições	Código trocado por <code>refreshToken</code> ; <i>profiles</i> listados; <i>profile</i> selecionado pelo marketplace da loja; <code>AmazonAdsConnection</code> criada/atualizada (<code>upsert</code>) com <code>ProfileId</code> , <code>CountryCode</code> , <code>CurrencyCode</code> e <code>IsActive = true</code> .

Contrato de Operação — UC-03

Contrato	
Operação	<code>Task desconectarLoja(Guid storeId)</code>
Referências cruzadas	UC-03 (reverte UC-01)
Pré-condições	Usuário autenticado e dono da loja; existe <code>AmazonConects</code> ativo vinculado à <code>storeId</code> .
Pós-condições	<code>AmazonConects.Disconnect()</code> aplicado (<code>IsActive = false</code> , tokens limpos); <code>subscriptions</code> de notificação canceladas na SP-API (best-effort); estado persistido no banco central.

3. Modelos de Projeto

3.1 Arquitetura

O Hermes adota **Monólito Modular + Clean Architecture + CQRS + Outbox**.

- Monólito Modular:** solução única (`Amazon.sln`) com nove módulos isolados em `Modules/` e código transversal em `Shareds/`. O *deploy* é simples (uma API), mas a fronteira entre *bounded contexts* é explícita — cada módulo tem seus projetos e seu banco. Justifica-se pela equipe enxuta e pela necessidade de evoluir contextos (`Stores`, `Payments`, `Products`) de forma independente sem o custo operacional de microsserviços.
- Clean Architecture por módulo:** cada módulo separa `Domain` (entidades, enums, interfaces de serviço), `Application` (`Commands/Queries` + `Handlers`), `Contracts` (DTOs) e `Infrastructure` (`DbContexts`, serviços concretos, `workers`). As dependências apontam para dentro (`Infrastructure` → `Application` → `Domain`), protegendo as regras de negócio de detalhes de infraestrutura.

- **CQRS com MediatR:** controllers apenas traduzem HTTP em Commands/Queries e despacham via `_mediator.Send(...)`. Comandos e consultas seguem caminhos distintos, com `ValidationBehavior<, >` (`FluentValidation`) interceptando o *pipeline*. Isso isola intenção de escrita e leitura — essencial num domínio com leituras analíticas pesadas (métricas de vendas/Ads) e escritas transacionais (sincronização de pedidos).
- **Outbox Pattern:** eventos de integração são persistidos em `out.OutBoxMessages` (`IEventBus` → `InMemoryEventBus`) na mesma transação do dado, e republicados de forma assíncrona pelo `OutBoxWorker` via `IMediator.Publish`. Garante entrega confiável de eventos (atomicidade dado+evento) sem acoplar os módulos a um broker síncrono.
- **BFF no Portal Web:** o front-end Next.js expõe rotas internas `/api/*` mapeadas por `BFF_ENDPOINTS`. Essas rotas executam no servidor, validam a sessão com `NextAuth`, adicionam o token JWT e encaminham as requisições para a API Host. Assim, a interface React não precisa expor diretamente a URL interna da API nem detalhes de autenticação.

C4 — Nível 1 (Contexto)

```
@startuml c4-contexto
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml
title C4 Nível 1 - Diagrama de Contexto do Sistema Hermes

Person(seller, "Vendedor Amazon", "Seller assinante que gere Ads, vendas e operacao da loja")
Person(admin, "Administrador de Conta", "Gere billing account e assinaturas")

System(hermes, "Hermes", "Plataforma de gestao para sellers Amazon: Ads, metricas, sincronizacao SP-API, assinatura e automacao")

System_Ext(spapi, "Amazon SP-API", "Pedidos, inventario, financeiro, listing, notificacoes")
System_Ext(ads, "Amazon Ads API", "Campanhas, keywords, relatorios de publicidade")
System_Ext(sqs, "AWS SQS / EventBridge", "Notificacoes assincronas da Amazon")
System_Ext(stripe, "Stripe", "Pagamentos e assinaturas")
System_Ext(legacy, "PagSeguro / Eduzz / Kiwify", "Gateways de pagamento legados")
System_Ext(keepa, "Keepa", "Historico e precos de mercado")
System_Ext(melhorenvio, "Melhor Envio", "Calculo de frete")
System_Ext(inpi, "Infosimples / INPI", "Consulta de marca/trademark")
System_Ext(azure, "Azure AI", "Vision / Document Intelligence")
System_Ext(brevo, "Brevo", "Envio de e-mail")
System_Ext(wapi, "W-API", "Notificacoes WhatsApp")

Rel(seller, hermes, "Usa", "HTTPS / Portal Web com BFF / Extensao Chrome")
Rel(admin, hermes, "Administra", "HTTPS")
Rel(hermes, spapi, "OAuth, le pedidos/financeiro, assina notificacoes")
Rel(hermes, ads, "OAuth, cria campanhas, baixa relatorios")
Rel(sqs, hermes, "Entrega notificacoes SP-API")
Rel(hermes, stripe, "Cria assinaturas; recebe webhooks")
Rel(hermes, legacy, "Recebe webhooks de assinatura")
Rel(hermes, keepa, "Consulta historico de produtos")
Rel(hermes, melhorenvio, "Calcula frete")
Rel(hermes, inpi, "Consulta marca")
Rel(hermes, azure, "Processa catalogos de fornecedor")
Rel(hermes, brevo, "Envia e-mails transacionais")
Rel(hermes, wapi, "Envia notificacoes WhatsApp")
@enduml
```

C4 — Nível 2 (Container)

```
@startuml c4-container
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
title C4 Nível 2 - Diagrama de Container do Sistema Hermes

Person(seller, "Vendedor Amazon", "Seller assinante")

System_Boundary(hermes, "Hermes") {
    Container(spa, "Portal Web UI", "Next.js 15 + React 19 + TypeScript + pnpm", "Interface do seller: dashboards de
```

```

Ads, conexao de loja, assinatura")
  Container(bff, "BFF do Portal", "Next.js API Routes + NextAuth", "Rotas /api/* validam sessao, ocultam URL interna e adicionam JWT server-side")
  Container(ext, "Extensao Chrome", "React 19 + TypeScript + Vite 6 + pnpm/Turbo", "Side panel, graficos na pagina de produto, cards de busca, calculadora e tarifas")
  Container(api, "API Host", "ASP.NET Core 9 (Monolito Modular)", "Controllers + MediatR (CQRS) + 9 modulos + workers internos")
  Container(outbox, "Worker Outbox", ".NET Worker", "Processa out.OutBoxMessages e republica eventos")
  Container(keepaw, "Keepa WorkeProcess", ".NET Worker", "CategorieWorker / SellersWorker")
  ContainerDb(sqlcentral, "SQL Server - Central/Modulos", "SQL Server", "StoresCentral, Usuarios, Produtos, Extensao, Planners, OutBox")
  ContainerDb(sqltenant, "SQL Server - Tenant por loja", "SQL Server", "Banco-por-loja (orders, products, financeiro, Ads facts)")
  ContainerDb(redis, "Redis", "Redis", "Cache de extensao/produtos/mineracao")
  ContainerQueue(rabbit, "RabbitMQ", "RabbitMQ", "Mensageria Products/Keepa")
}

System_Ext(spapi, "Amazon SP-API", "")
System_Ext(ads, "Amazon Ads API", "")
System_Ext(sqs, "AWS SQS", "")
System_Ext(stripe, "Stripe", "")

Rel(seller, spa, "Usa", "HTTPS")
Rel(seller, ext, "Usa", "HTTPS")
Rel(spa, bff, "Chama", "JSON/HTTPS /api/*")
Rel(bff, api, "Proxy autenticado", "JSON/HTTPS + Bearer JWT")
Rel(ext, api, "Chama", "JSON/HTTPS")
Rel(api, sqlcentral, "Le/escreve", "EF Core")
Rel(api, sqltenant, "Le/escreve por tenant", "EF Core")
Rel(api, redis, "Cacheia", "")
Rel(api, rabbit, "Publica/consume", "AMQP")
Rel(api, outbox, "Persiste eventos", "out.OutBoxMessages")
Rel(outbox, sqlcentral, "Le eventos", "EF Core")
Rel(api, spapi, "OAuth/REST")
Rel(api, ads, "OAuth/REST")
Rel(sqs, api, "Notificacoes")
Rel(api, stripe, "REST + webhooks")
Rel(keepaw, rabbit, "Publica", "AMQP")
@enduml

```

3.2 Diagrama de Componentes e Implantação

Componentes (módulos + infraestrutura)

```

@startuml componentes
title Diagrama de Componentes - Portal BFF + API Host
skinparam componentStyle rectangle

package "Portal Web (Next.js)" {
  [React UI] as P_UI
  [BFF_ENDPOINTS (/api/*)] as P_ENDPOINTS
  [API Routes] as P_ROUTES
  [NextAuth / Session] as P_AUTH
}

package "Host (Presentation)" {
  [Controllers] as CTRL
  [Configs / DI] as CFG
}

```



```

package "Shareds" {
  [Host.Domain (BaseEntity)] as SD
  [Host.Contracts (IEventBus / Eventos)] as SC
  [Host.Infrastructure (UserContext, Behaviours)] as SI
  [AmazonAds.SDK] as ADSSDK
  [PagSeguro.SDK] as PGSDK
  [Keepa.SDK] as KPSDK
  [MelhorEnvio.SDK] as MESDK
  [Infosimples.SDK] as INSDK
}

```

```

package "Modulos" {
  [Usuarios] as M_USERS
  [Payments] as M_PAY
  [GatewayPay] as M_GW
  [Stores] as M_STORES
  [Products] as M_PROD
  [ExtensaoAPI] as M_EXT
  [Planner] as M_PLAN
  [Keepa] as M_KEEPA
  [OutBox] as M_OUT
}

```

```

database "SQL Server\n(Central + Modulos)" as DBC
database "SQL Server\n(Tenant por loja)" as DBT
database "Redis" as REDIS
queue "RabbitMQ" as RMQ

```

```

P_UI --> P_ENDPOINTS : fetch /api/*
P_ENDPOINTS --> P_ROUTES
P_ROUTES --> P_AUTH : valida sessao
P_ROUTES --> CTRL : HTTPS + Bearer JWT

```

```

CTRL --> M_USERS : MediatR Send
CTRL --> M_PAY : MediatR Send
CTRL --> M_GW : MediatR Send
CTRL --> M_STORES : MediatR Send
CTRL --> M_PROD : MediatR Send
CTRL --> M_EXT : MediatR Send
CTRL --> M_PLAN : MediatR Send
CFG ..> SI
CFG ..> SC

```

```

M_STORES --> ADSSDK
M_GW --> PGSDK
M_KEEPA --> KPSDK
M_EXT --> MESDK
M_EXT --> INSDK

```

```

M_USERS --> DBC
M_PAY --> DBC
M_PROD --> DBC
M_EXT --> DBC
M_PLAN --> DBC
M_STORES --> DBC : central (AmazonConects, Stores)
M_STORES --> DBT : tenant (orders, facts)
M_EXT --> REDIS
M_PROD --> REDIS
M_PROD --> RMQ
M_KEEPA --> RMQ

```

```

SC <.. M_OUT : persiste eventos

```

```

M_OUT --> DBC : out.OutBoxMessages
M_USERS ..> SD
M_STORES ..> SD
@enduml

```

Implantação (Docker / Swarm)

```

@startuml implantacao
title Diagrama de Implantacao - Hermes (Docker / Swarm)
skinparam node {
    BackgroundColor #FDFDFD
}

node "Cluster Docker Swarm" {
    node "Servico: hosts" as N_API {
        artifact "API Host\nASP.NET Core 9\n(8080/8081)\n2 vCPU / 1 GB" as A_API
        artifact "Workers internos\n(OrderSync, Pricing,\nAmazonAdsReport, ...)" as A_WRK
    }
    node "Servico: hosts_outbox_workeprocess" as N_OUT {
        artifact "Outbox Worker\n1 vCPU / 512 MB" as A_OUT
    }
    node "Servico: keepa workeprocess" as N_KEEPA {
        artifact "Keepa WorkeProcess" as A_KEEPA
    }
    node "Servico: redis" as N_REDIS {
        artifact "Redis (appendonly)\n6379\n0.5 vCPU / 512 MB" as A_REDIS
    }
    node "Servico: rabbitmq" as N_RMQ {
        artifact "RabbitMQ" as A_RMQ
    }
}

node "SQL Server (externo)" as N_SQL {
    database "Bancos: StoresCentral,\nUsuarios, Produtos, Extensao,\nPlanners, OutBox + bancos tenant" as DB
}

cloud "APIs externas" as CLOUD {
    artifact "Amazon SP-API / Ads"
    artifact "AWS SQS"
    artifact "Stripe / Gateways"
}

A_API --> A_REDIS : TCP 6379
A_API --> A_RMQ : AMQP
A_API --> DB : TCP 11433 (EF Core)
A_OUT --> DB
A_KEEPA --> A_RMQ
A_API --> CLOUD : HTTPS
A_WRK --> CLOUD : HTTPS
@enduml

```

3.3 Diagrama de Classes

```

@startuml classes
title Diagrama de Classes - Dominio Hermes (entidades-chave)
skinparam classAttributeIconSize 0

abstract class BaseEntity {
    +Id : Guid
}

```

```

+CreatedAt : DateTime
+CreatedBy : string
+UpdatedAt : DateTime
+UpdateBy : string
+RowVersion : byte[]
}

class AmazonConects {
+Provider : string
+SellingPartnerId : string
+RefreshToken : string
+AccessToken : string
+MwsAuthToken : string
+MarketplaceId : string
+IsActive : bool
+SyncStatus : SyncStatus
+ConsecutiveErrorCount : int
--
+New() : AmazonConects
+Disconnect()
+RecordError(msg)
+TouchSync()
+TransitionSyncStatus(s)
}

class Stores {
+UserId : Guid
+StoreName : string
+ConnectId : Guid
+ReviewSolicitationEnabled : bool
--
+New() : Stores
}

class AmazonAdsConnection {
+UserId : Guid
+StoreId : Guid
+ProfileId : string
+RefreshToken : string
+CountryCode : string
+CurrencyCode : string
+IsActive : bool
}

class StoreTenant {
+StoreName : string
+TenantKey : string
+ConnectionString : string
+StoreId : Guid
+IsActive : bool
}

class Order {
+AmazonOrderId : string
+Status : string
+FulfillmentChannel : string
+OrderTotalAmount : decimal
+PurchaseDateUtc : DateTime
}

class OrderItem {
+AmazonOrderItemId : string

```

```

+Asin : string
+Quantity : int
+ItemPrice : decimal
}

class OrderFinancialSummary {
+EstimatedRevenue : decimal
+EstimatedProfit : decimal
+RealRevenue : decimal
+RealProfit : decimal
+FinancialStatus : string
}

class Product {
+Asin : string
+SellerSku : string
+MarketplaceId : string
+Title : string
+CurrentPrice : decimal
}

class PricingStrategy {
+StrategyType : string
+IsActive : bool
}

class AmazonAdsReportJob {
+ReportTypeId : string
+Status : string
+NextPollAtUtc : DateTime
}

class Users {
+Name : string
+Email : string
+TaxId : string
+Status : UserStatus
}

class Subscription {
+Provider : string
+Status : StatusSubscription
+ExpirationDate : DateTime
+AutoRenew : bool
--
+New()
+SetStatus(s)
+Deactivate()
+Reactivate()
}

class Plans {
+Name : string
+Price : decimal
}

class PaymentSubscription {
+Provider : string
+Status : PaymentSubscriptionStatus
+CurrentPeriodEndUtc : DateTime
+CancelAtPeriodEnd : bool
--

```

```

+CreateStripe()
+MarkPaid()
+MarkTrialing()
+Cancel()
}

class BillingAccount {
+Type : BillingAccountType
+Status : BillingAccountStatus
}

class OutBoxMessage {
+Message : string
+Type : string
+SavedOn : DateTime
+ExecutedOn : DateTime
}

BaseEntity <|-- AmazonConects
BaseEntity <|-- Stores
BaseEntity <|-- AmazonAdsConnection
BaseEntity <|-- StoreTenant
BaseEntity <|-- Subscription

Stores "1" --> "1" AmazonConects : ConnectId
Stores "1" --> "0..1" AmazonAdsConnection : StoreId
Stores "1" --> "1" StoreTenant : StoreId
Order "1" *-- "many" OrderItem
Order "1" --> "1" OrderFinancialSummary
OrderItem "many" --> "0..1" Product
Users "1" --> "many" Subscription
Subscription "many" --> "1" Plans
BillingAccount "1" --> "many" PaymentSubscription
@enduml

```

3.4 Diagramas de Sequência

Sequência de projeto — UC-01

```

@startuml seq-uc01
title Sequencia de Projeto UC-01 - Conexao Amazon SP-API
actor Vendedor
participant "StoreConnectionController" as CTRL
participant "Mediatr" as MED
participant "AuthorizeAmazonCommandHandler" as H
participant "IAmazonOAuthService" as OAUTH
participant "IAmazonConnectService" as CONN
participant "IAmazonSellerApiService" as SELLER
participant "IStoreService" as STORE
participant "IAmazonNotificationService" as NOTIF
database "StoreCentralDbContext" as DB
participant "Amazon SP-API" as SPAPI

Vendedor -> CTRL : GET /AuthorizeAmazon(code, state)
CTRL -> CTRL : ParseState(state) -> userId
CTRL -> MED : Send(AuthorizeAmazonCommand)
MED -> H : Handle(command)
H -> OAUTH : ExchangeAuthorizationCodeAsync(code)
OAUTH -> SPAPI : POST /auth/o2/token
SPAPI --> OAUTH : tokens

```

```

OAUTH --> H : refreshToken, accessToken
H -> CONN : UpsertConnectionAsync(...)
CONN -> DB : save AmazonConects
DB --> CONN : connectId
CONN --> H : connectId
H -> SELLER : GetSellerInfoAsync(refreshToken)
SELLER -> SPAPI : GET seller info
SPAPI --> SELLER : sellerInfo
SELLER --> H : sellerInfo
H -> STORE : CreateAsync(userId, storeName, connectId)
STORE -> DB : save Stores
H -> NOTIF : EnsureSubscriptionAsync(ORDER_CHANGE)
NOTIF -> SPAPI : create subscription
SPAPI --> NOTIF : subscriptionId
H --> MED : "https://portal.hermess.app"
MED --> CTRL : redirectUrl
CTRL --> Vendedor : 302 Redirect
@enduml

```

Sequência de projeto — UC-02

```

@startuml seq-uc02
title Sequencia de Projeto UC-02 - Conexao Amazon Ads
actor Vendedor
participant "AmazonAdsConnectionController" as CTRL
participant "Mediatr" as MED
participant "AuthorizeAmazonAdsCommandHandler" as H
participant "IAmazonAdsAuthService" as AUTH
participant "IAmazonAdsProfilesClient" as PROF
participant "IAmazonAdsConnectionService" as CONN
database "StoreCentralDbContext" as DB
participant "Amazon Ads API" as ADS

Vendedor -> CTRL : GET /Authorize(code, state)
CTRL -> MED : Send(AuthorizeAmazonAdsCommand)
MED -> H : Handle(command)
H -> AUTH : ExchangeAuthorizationCodeAsync(code)
AUTH -> ADS : POST /auth/o2/token
ADS --> AUTH : refreshToken
AUTH --> H : refreshToken
H -> PROF : ListProfilesAsync(refreshToken)
PROF -> ADS : GET /v2/profiles
ADS --> PROF : profiles[]
PROF --> H : profiles[]
H -> H : seleciona profile por marketplace da loja
H -> CONN : UpsertAsync(AmazonAdsConnection)
CONN -> DB : save AmazonAdsConnection
DB --> CONN : ok
H --> MED : resultado
MED --> CTRL : resultado
CTRL --> Vendedor : 200 / redirect
@enduml

```

Sequência de projeto — UC-03

```

@startuml seq-uc03
title Sequencia de Projeto UC-03 - Desconectar loja
actor Vendedor
participant "StoreConnectionController" as CTRL

```

```

participant "MediatR" as MED
participant "DisconnectStoreCommandHandler" as H
participant "AmazonConects" as AGG
database "StoreCentralDbContext" as DB
participant "IAmazonNotificationService" as NOTIF
participant "Amazon SP-API" as SPAPI

```

```

Vendedor -> CTRL : PATCH /{storeId}/Disconnect
CTRL -> MED : Send(DisconnectStoreCommand)
MED -> H : Handle(command)
H -> DB : busca Stores + AmazonConects por storeId
DB --> H : conexao
H -> AGG : Disconnect()
note right of AGG : IsActive=false\nlimpa tokens
H -> NOTIF : cancela subscriptions (best-effort)
NOTIF -> SPAPI : delete subscriptions
SPAPI --> NOTIF : ok
H -> DB : SaveChanges
H --> MED : ok
MED --> CTRL : ok
CTRL --> Vendedor : 200 OK
@enduml

```

3.5 Diagramas de Comunicação

Comunicação — UC-01

```

@startuml comunicacao-uc01
title Diagrama de Comunicacao UC-01 - Conexao Amazon SP-API
left to right direction

```

```

actor Vendedor as V
object "StoreConnectionController" as CTRL
object "AuthorizeAmazonCommandHandler" as H
object "IAmazonOAuthService" as OAUTH
object "IAmazonConnectService" as CONN
object "IAmazonSellerApiService" as SELLER
object "IStoreService" as STORE
object "IAmazonNotificationService" as NOTIF
database "StoreCentralDbContext" as DB

V --> CTRL : 1: AuthorizeAmazon(code, state)
CTRL --> H : 2: Send(AuthorizeAmazonCommand)
H --> OAUTH : 3: ExchangeAuthorizationCode(code)
H --> CONN : 4: UpsertConnection() -> connectId
CONN --> DB : 4.1: save AmazonConects
H --> SELLER : 5: GetSellerInfo(refreshToken)
H --> STORE : 6: CreateAsync(userId, connectId)
STORE --> DB : 6.1: save Stores
H --> NOTIF : 7: EnsureSubscription(ORDER_CHANGE)
H --> CTRL : 8: redirectUrl
CTRL --> V : 9: 302 Redirect
@enduml

```

3.6 Diagramas de Estados

Estado — Campanha Amazon Ads

```
@startuml estado-campanha
title Diagrama de Estados - Campanha Amazon Ads
[*] --> Rascunho : monta campanha (frontend)
Rascunho --> Ativa : CreateCampaign (State=ENABLED)
Ativa --> Pausada : UpdateCampaign (State=PAUSED)
Pausada --> Ativa : UpdateCampaign (State=ENABLED)
Ativa --> Encerrada : DeleteCampaign (State=ARCHIVED)
Pausada --> Encerrada : DeleteCampaign (State=ARCHIVED)
Encerrada --> [*]

note right of Ativa
    Estrategias automaticas (UC-07)
    ajustam bids enquanto ENABLED
end note
@enduml
```

Estado — Conexão de Loja

```
@startuml estado-conexao-loja
title Diagrama de Estados - Conexao de Loja (AmazonConnects)
[*] --> Desconectada
Desconectada --> Conectando : New() / InitiateAmazon
Conectando --> Conectada : AuthorizeAmazon (tokens validos, IsActive=true)
Conectando --> Desconectada : falha OAuth

state Conectada {
    [*] --> NotStarted
    NotStarted --> SyncingInventory
    SyncingInventory --> SyncingProducts
    SyncingProducts --> SyncingOrders
    SyncingOrders --> SyncingFinancial
    SyncingFinancial --> ProcessingBacklog
    ProcessingBacklog --> Ready
    Ready --> Ready : notificacoes processadas
}

Conectada --> Expirada : RecordError (>=2 erros) / token expirado
Expirada --> Conectada : TouchSync (re-auth, zera erros)
Conectada --> Desconectada : Disconnect() (UC-03)
Expirada --> Desconectada : Disconnect()
Desconectada --> [*]
@enduml
```

A submáquina interna de Conectada reflete o enum SyncStatus (NotStarted → SyncingInventory → SyncingProducts → SyncingOrders → SyncingFinancial → ProcessingBacklog → Ready). Notificações só são processadas quando SyncStatus == Ready.

4. Modelos de Dados

Diagrama Entidade-Relacionamento

```
@startuml modelo-dados-er
title Modelo de Dados (ER) - Hermes
hide circle
skinparam linetype ortho

' ===== Banco central (StoresCentral) =====
entity "AmazonConnects" as connects {
    *Id : uniqueidentifier <<PK>>
    --
    SellingPartnerId : nvarchar
    MarketplaceId : nvarchar
    RefreshToken : nvarchar(2048)
    AccessToken : nvarchar(2048)
    IsActive : bit
    SyncStatus : int
    ConsecutiveErrorCount : int
    ' UNIQUE (SellingPartnerId, MarketplaceId)
}

entity "Stores" as stores {
    *Id : uniqueidentifier <<PK>>
    --
    UserId : uniqueidentifier
    StoreName : nvarchar
    ConnectId : uniqueidentifier <<FK>>
    ReviewSolicitationEnabled : bit
}

entity "AmazonAdsConnections" as adsconn {
    *Id : uniqueidentifier <<PK>>
    --
    UserId : uniqueidentifier
    StoreId : uniqueidentifier <<FK>>
    ProfileId : nvarchar
    ' UNIQUE (UserId, ProfileId)
}

entity "StoreTenants" as tenant {
    *Id : uniqueidentifier <<PK>>
    --
    StoreId : uniqueidentifier <<FK>>
    ConnectionString : nvarchar(2048)
    IsActive : bit
}

entity "StoreTaxConfigurations" as tax {
    *Id : uniqueidentifier <<PK>>
    --
    StoreId : uniqueidentifier
    RegimeTributario : int
    DAS : decimal(18,4)
    IsActive : bit
}

' ===== Banco tenant (por loja) =====
entity "products" as products {
```

```

    *Id : uniqueidentifier <<PK>>
    --
    Asin : nvarchar
    SellerSku : nvarchar
    MarketplaceId : nvarchar
    CurrentPrice : decimal(18,2)
    ' UNIQUE (Asin, SellerSku, MarketplaceId)
}

entity "orders" as orders {
    *Id : uniqueidentifier <<PK>>
    --
    AmazonOrderId : nvarchar <<UQ>>
    Status : nvarchar
    OrderTotalAmount : decimal(18,2)
}

entity "order_items" as items {
    *Id : uniqueidentifier <<PK>>
    --
    OrderId : uniqueidentifier <<FK>>
    ProductId : uniqueidentifier <<FK>>
    AmazonOrderItemId : nvarchar
}

entity "order_financial_summaries" as fin {
    *Id : uniqueidentifier <<PK>>
    --
    OrderId : uniqueidentifier <<FK,UQ>>
    EstimatedProfit : decimal(18,2)
    RealProfit : decimal(18,2)
    FinancialStatus : nvarchar
}

entity "amazon_ads_report_jobs" as adsjob {
    *Id : uniqueidentifier <<PK>>
    --
    StoreId : uniqueidentifier
    ReportTypeId : nvarchar
    Status : nvarchar
}

entity "amazon_ads_advertised_product_daily_facts" as adsfact {
    *Id : uniqueidentifier <<PK>>
    --
    ReportJobId : uniqueidentifier <<FK>>
    Asin : nvarchar
    Spend : decimal(18,2)
    AdSales : decimal(18,2)
}

' ===== Banco Usuarios =====
entity "Users" as users {
    *Id : uniqueidentifier <<PK>>
    --
    Email : nvarchar
    TaxId : nvarchar
    Status : int
}

entity "Subscriptions" as subs {
    *Id : uniqueidentifier <<PK>>

```

```

--
UserId : uniqueidentifier <<FK>>
PlanId : uniqueidentifier <<FK>>
Status : int
}

entity "Plans" as plans {
  *Id : uniqueidentifier <<PK>>
  --
  Name : nvarchar
  Price : decimal(18,2)
}

' ===== Banco OutBox (schema out) =====
entity "out.OutBoxMessages" as outbox {
  *Id : uniqueidentifier <<PK>>
  --
  Message : nvarchar(max)
  Type : nvarchar
  SavedOn : datetime2
  ExecutedOn : datetime2
}

connects ||--o{ stores : ConnectId
stores ||--o| adsconn : StoreId
stores ||--|| tenant : StoreId
stores ||--o{ tax : StoreId
orders ||--|{ items : OrderId
products ||--o{ items : ProductId
orders ||--|| fin : OrderId
adsjob ||--o{ adsfact : ReportJobId
users ||--o{ subs : UserId
plans ||--o{ subs : PlanId
@enduml

```

Estratégia de mapeamento objeto-relacional (EF Core)

- **EF Core 9 sobre SQL Server** (UseSqlServer). O mapeamento é feito por **Fluent API** no `OnModelCreating` de cada `DbContext` (não há classes `IEntityTypeConfiguration` separadas, exceto `OutBoxMessageEntityConfiguration`).
- **Conversão de enums:** enums de estado são persistidos como inteiro (`HasConversion<int>().ValueGeneratedNever()`), por exemplo `AmazonConnects.SyncStatus` e `StoreTaxConfiguration.RegimeTributario`. Em serialização HTTP, os enums são expostos como string (`JsonStringEnumConverter`).
- **Value objects / owned types:** os agregados encapsulam estado com construtor privado
 - `factory New(...)` e métodos de transição (`Disconnect`, `TouchSync`, `TransitionSyncStatus`), seguindo DDD tático. A base comum `BaseEntity` concentra `Id`, auditoria (`CreatedAt/CreatedBy/UpdatedAt/UpdateBy`) e `RowVersion`.
- **Schemas por módulo:** o módulo `OutBox` usa o schema `out` (`out.OutBoxMessages`); os demais usam `dbo`. A separação física principal, porém, é por **banco de dados distinto por módulo** (`StoresCentral`, `Usuarios`, `Produtos`, `Extensao`, `Planners`, `OutBox`).
- **Multi-tenant banco-por-loja:** o `StoreTenantDbContext` é o mesmo schema replicado em N bancos de loja, instanciado dinamicamente com a `ConnectionString` registrada em `StoreTenants`.
- **Convenções de nomenclatura:** tabelas centrais em **PascalCase** (`AmazonConnects`, `Stores`, `StoreTenants`); tabelas tenant em **snake_case** (`products`, `orders`, `order_items`, `financial_events`).
- **Tipos monetários:** `decimal(18,2)` para valores e `decimal(18,4)` para alíquotas/bids.
- **Auditoria automática:** `SaveChangesAsync` preenche os campos de auditoria de `BaseEntity` (default "system"). No banco central, `CreatedBy/UpdateBy` são *nullable* para suportar o fluxo OAuth sem *user context*.
- **Concorrência otimista:** `IsRowVersion()` em colunas `rowversion` (ex.: `InventoryResyncLease`).
- **Migrations:** geridas por módulo (`Usuarios: 9`, `ExtensaoAPI: 16`, `Products: 10`, `Planner: 4`, `Payments: 2`, `OutBox: 1`). O módulo **Stores não possui migrations** — o schema central/tenant é criado e ajustado em runtime via `Database.Migrate()`, reparo de schema (`ReviewAutomationSettingsSchemaRepair`) e `TenantMigrationWorker`.